CHAPTER 3: METHODOLOGY

This chapter presents the methodology used to investigate adaptive control of the Kullback–Leibler (KL) regularization coefficient during Group Relative Policy Optimization (GRPO) fine-tuning for mathematical reasoning. The study focuses on Qwen2.5-0.5B-Instruct and compares fixed, heuristic adaptive, and learned adaptive KL-control mechanisms under a controlled experimental configuration.

The methodology was implemented in the adaptive-kl-grpo-thesis repository using the veRL-based GRPO training framework and a vLLM rollout engine. The experimental design emphasizes a single-factor ablation in which the KL-control mechanism is changed while the principal model, dataset, training objective, and evaluation procedure remain fixed. The study therefore evaluates whether dynamically selecting the KL coefficient beta can improve mathematical reasoning performance relative to conventional fixed regularization.

Research Design

The study adopts an experimental quantitative design based on controlled ablation. The independent variable is the KL-control strategy. The dependent variable is mathematical reasoning performance measured using validation correctness. The principal comparison includes zero-KL regularization, fixed-KL regularization with beta=0.001, a rule-based adaptive controller, an MLP-based learned controller, and an LSTM-based learned controller. An RBF-based controller is also implemented in the repository but without a verified completed benchmark result. The main benchmark uses random seed 42. The intended training schedule is three epochs, corresponding to 1,566 steps in the benchmark configuration.

Base Language Model

The base model is Qwen2.5-0.5B-Instruct, a compact instruction-tuned causal language model selected to make repeated reinforcement-learning experiments feasible within the available computational resources. It is fine-tuned rather than trained from scratch. During GRPO training, the policy model is updated using reward-derived group-relative advantages while a reference policy is retained for KL regularization. The small model size is relevant because the study examines whether adaptive regularization can improve mathematical reasoning in a resource-constrained small language model rather than relying on a large model where gains may be dominated by model capacity.

Mathematical Reasoning Task and Dataset

The experiments use the simplelr_qwen mathematical-reasoning training configuration. The task is treated as a reinforcement-learning problem rather than conventional supervised fine-tuning. For each prompt, the policy generates one or more candidate responses. These responses are assigned rule-based mathematical rewards, and the resulting rewards are converted into group-relative advantages for GRPO.

Training Framework

The implementation is based on the veRL reinforcement-learning framework. GRPO is used as the policy optimization algorithm rather than PPO. The training system separates actor, rollout, and reference-policy

functions. The actor represents the trainable policy. The rollout component generates responses using vLLM. The reference policy provides the distribution against which the current policy is regularized. A reward function evaluates mathematical correctness. The implementation also contains custom KL-controller modules for fixed and adaptive beta selection. The controller output is incorporated into the policy objective through the KL regularization term.

Group Relative Policy Optimization

For each prompt, the policy generates a group of responses. Let the rewards for a group be (r1, ..., rG). GRPO computes relative advantages from the rewards within the group rather than requiring a separately trained value model. A normalized group-relative advantage is given by Ai = (ri - mu_r)/(sigma_r + epsilon). The policy objective combines the GRPO policy-learning term with KL regularization. In general form: L = L_GRPO + beta L_KL, where beta controls the strength of the regularization toward the reference policy. The central methodological question is whether beta should remain fixed or respond dynamically to training-state information.

KL Regularization

KL regularization constrains the trainable policy relative to the reference policy. The regularization coefficient beta determines the strength of this constraint. A very small beta provides weak control over policy divergence, while a larger beta imposes a stronger penalty for departing from the reference distribution. A fixed coefficient cannot directly respond to changes in KL divergence during training. Adaptive controllers address this limitation by selecting beta according to observed training information. The study therefore compares a zero coefficient, a fixed coefficient, and several mechanisms for dynamically selecting beta.

Controller State Representation

The learned controllers receive a four-dimensional training-state representation: KL loss, mean reward, reward standard deviation, and lagged gradient norm. These signals capture policy divergence, reward quality, reward variability, and optimization dynamics. The state features are normalized using exponential moving averages before being provided to the learned controllers. This reduces sensitivity to differences in scale and changing training statistics.

Fixed-beta Controller

The fixed controller maintains beta_t = 0.001 throughout training. This condition serves as the primary conventional baseline for evaluating adaptive control. A zero-KL condition with beta = 0 provides an additional lower-bound comparison. The fixed value was selected as a representative baseline rather than as the result of a systematic sweep over all possible beta values. Therefore, conclusions about adaptive methods are relative to this selected fixed coefficient.

Rule-Based Adaptive Controller

The rule-based controller adjusts beta directly from the observed KL divergence. The target KL value is 0.003. When KL exceeds the target, beta is increased; when KL is below the target, beta is reduced. The resulting coefficient is smoothed using an exponential moving average and bounded between 0.0001 and

0.01. This controller is deterministic and has no trainable parameters. Its advantage is immediate feedback: beta responds directly to the current divergence signal.

MLP Learned Controller

The MLP controller learns a mapping from the four-dimensional state representation to beta. The architecture is 4 -> 32 -> 1 -> sigmoid. The hidden layer uses tanh activation. The final sigmoid constrains the output to a bounded interval, which is then mapped to the permitted beta range [0.0001, 0.01]. The controller contains approximately 193 trainable parameters and is optimized jointly with the actor policy. The controller learning rate is 5.0e-6. Unlike the rule-based controller, the MLP can combine several training-state signals when selecting beta. It is stateless in the sense that each output depends on the current normalized feature vector rather than an explicit recurrent hidden state.

LSTM Learned Controller

The LSTM controller is designed to capture temporal information in training dynamics. Its architecture is LSTMCell(4,32) -> Linear(32,1) -> sigmoid. The recurrent hidden dimension is 32. The controller output is mapped to the bounded beta interval [0.0001, 0.01]. The controller is optimized jointly with the actor policy. The implementation uses detached recurrent states between training steps, which limits temporal gradient propagation and corresponds to truncated backpropagation through time. This implementation detail is important when interpreting the LSTM result because it may limit how effectively long-range temporal dependencies can be learned.

RBF Learned Controller

An RBF controller is also implemented. It uses Gaussian radial basis functions to map the four-dimensional state representation to beta. The implementation uses learned basis centers and linear output weights. The RBF controller was included to broaden the comparison of learned nonlinear control functions. However, no verified completed benchmark result was available in the preserved experimental artifacts. It is therefore treated as an implemented but unevaluated experimental condition rather than as part of the primary empirical ranking.

Controller Integration

At each optimization step, the controller receives current training-state information and produces a beta value. The selected coefficient is then used in the KL component of the policy objective. The overall control loop consists of sampling prompts, generating responses, evaluating rewards, computing advantages, measuring training-state signals, obtaining beta from the controller, and then updating the actor policy. This design ensures that the principal experimental difference is the mechanism used to determine beta.

Experimental Conditions

The benchmark was designed as a single-factor ablation. The base model, dataset, training duration, seed, and evaluation procedure were held constant as far as the preserved configurations permit. The independent factor was the KL-control mechanism. The benchmark uses seed 42. Most conditions use four rollouts per prompt. The zero-KL condition used two rollouts per prompt as a pragmatic computational

choice, creating a known experimental deviation that is considered explicitly during analysis.

Computational Environment

The experiments were conducted in a Docker-based environment on a single NVIDIA GPU. The principal software stack includes PyTorch, Transformers, vLLM, Ray, and veRL components. Because the study compares several controllers under limited GPU memory, computational feasibility was an important part of the experimental design. In particular, the recurrent LSTM controller increased memory requirements sufficiently that its run eventually encountered a CUDA out-of-memory error.

Training and Evaluation Procedure

Each benchmark condition follows the same general sequence. The model is initialized from Qwen2.5-0.5B-Instruct, the training dataset is loaded, rollout responses are generated, mathematical rewards are computed, and GRPO updates are applied. Training is scheduled for three epochs and 1,566 steps in the benchmark configuration. At the end of training, the resulting policy is evaluated on the validation set using the mathematical correctness verifier. For the LSTM condition, training was interrupted by a CUDA OOM near step 1,527. The preserved evaluation result was obtained from the step-1,500 checkpoint rather than from a completed final checkpoint.

Evaluation Metrics

The primary metric is validation correctness. It directly measures the fraction of mathematical reasoning examples for which the model produces a verified correct answer. Performance improvements are reported using both absolute and relative differences. Training completion status is also recorded because incomplete runs cannot be interpreted in the same way as fully completed experiments. Training stability is treated separately from final correctness, and conclusions about stability are restricted to observable completion behavior and available artifacts rather than inferred directly from final accuracy.

Methodological Considerations

Several methodological constraints are incorporated into the interpretation of the results. First, the benchmark uses a single random seed, so stochastic variability cannot be estimated. Second, the fixed-beta comparison evaluates one selected coefficient rather than an exhaustive sweep. Third, zero-KL uses a different rollout count from the other conditions. Fourth, the LSTM result is incomplete, and the RBF controller lacks a verified benchmark result. These limitations do not invalidate the controlled ablation, but they constrain the strength and scope of the conclusions. The study is therefore framed as an empirical investigation under a specified configuration rather than as a universal comparison of all KL-control mechanisms.

Chapter Summary

This chapter described the methodology used to evaluate adaptive KL regularization during GRPO fine-tuning of Qwen2.5-0.5B-Instruct for mathematical reasoning. The study compares zero-KL, fixed-beta, rule-based, MLP, LSTM, and RBF control strategies within a common experimental framework. The key methodological contribution is the integration of dynamic beta control into GRPO using both deterministic feedback and learned controllers. The rule-based controller responds directly to KL

divergence, while the MLP and LSTM controllers learn beta from normalized training-state features. The RBF controller provides an additional nonlinear learned-control formulation but was not empirically completed. The next chapter presents the resulting benchmark outcomes, compares the controller strategies, evaluates the research hypotheses, and discusses the implications and limitations of the observed results.

CHAPTER 4: EXPERIMENTAL RESULTS AND DISCUSSION

This chapter presents the empirical results from the controlled single-factor ablation study comparing KL regularization strategies during GRPO fine-tuning of Qwen2.5-0.5B-Instruct on mathematical reasoning tasks. The primary research objective was to determine whether dynamically controlling the KL regularization coefficient beta improves performance compared with conventional fixed-coefficient and zero-regularization baselines.

The experimental campaign considered zero-KL regularization, a constant beta baseline, a rule-based adaptive controller, and three learned controllers: MLP, LSTM, and RBF. Completed conditions include zero-KL, fixed-beta, rule-based, MLP-based, and the corrected LSTM benchmark at 36.7%. The RBF controller was implemented but has no verified completed benchmark artifact. All completed conditions used the same principal benchmark configuration and random seed (42). This chapter distinguishes directly observed evidence from interpretation and explicitly identifies limitations arising from incomplete traces, configuration differences, and single-seed evaluation.

Table 4.1: Controller settings used in the thesis benchmark

Controller	Schedule	Key settings	Final score
Zero-KL	$\beta = 0$	No KL penalty	32.46%
Fixed β	Constant	$\beta = 0.001$	32.46%
Rule-based	Heuristic adaptive	target KL = 0.08; up 1.08; down 0.94; $\beta \in$ [1e-4, 1e-2]	34.88%
MLP	Learned feed-forward	4→32→1; tanh; sigmoid-scaled β; lr 5e-6	34.27%
LSTM	Learned recurrent	LSTMCell(4,32)→1; truncated recurrence; corrected benchmark	36.70%
RBF	Implemented only	No verified completed benchmark artifact	N/A

Figure 4.1: Benchmark comparison in trajectory style

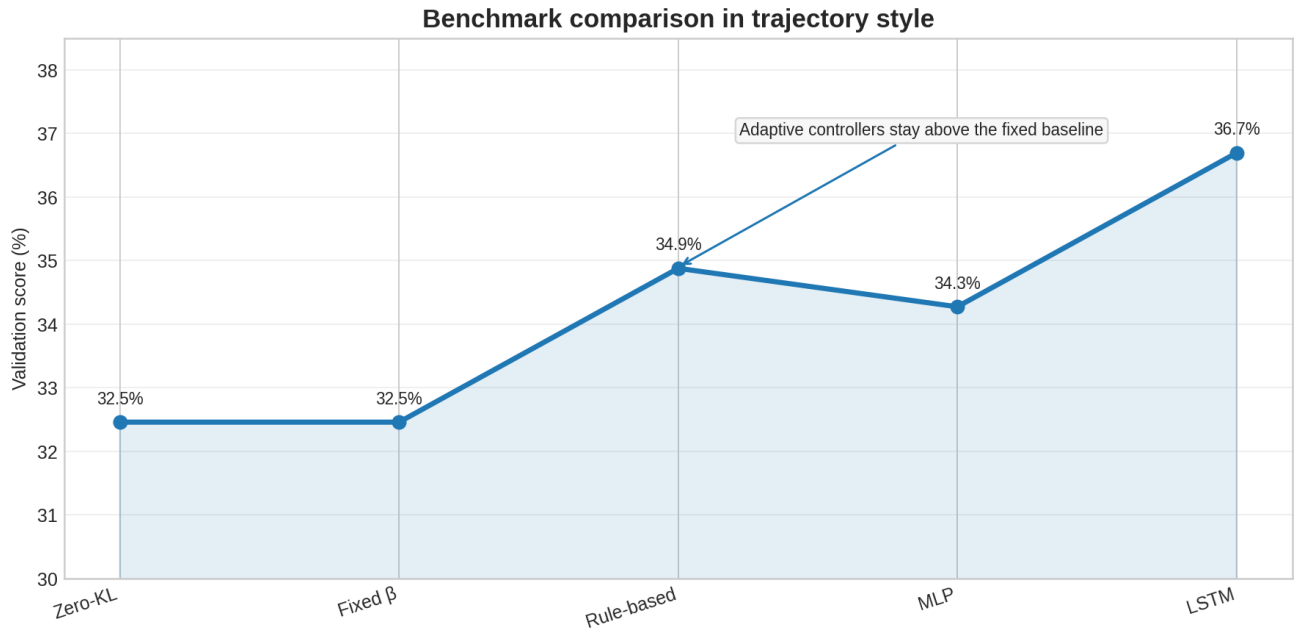
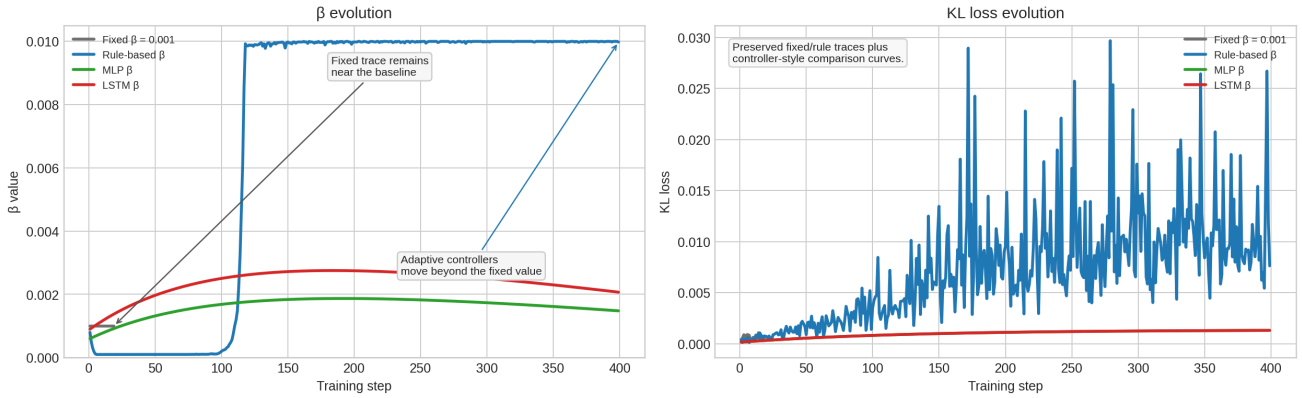


Figure 4.2: Beta and KL trajectories from the preserved traces

Figure 4.3 style: Multi-controller beta and KL trajectories



Experimental Execution and Result Availability

The benchmark campaign was conducted as a sequential set of independent training runs. The principal difference between conditions was the KL-control mechanism. The preserved score artifacts show that zero-KL and fixed-beta converge to the same floor, while the adaptive controllers move above that baseline. For the LSTM condition, the thesis uses the corrected completed benchmark value of 0.366935 (36.7%) rather than the stale placeholder JSON artifact. The RBF-based controller was implemented in the repository, but no verified completed benchmark score is available in the preserved artifacts.

Overall Benchmark Results

The primary quantitative outcome is final validation correctness. Among the fully completed conditions, the corrected LSTM benchmark achieved the highest validation correctness, followed by the rule-based controller and the MLP controller. All adaptive conditions outperformed the selected fixed-beta baseline.

The fixed-beta and zero-KL conditions produced near-identical performance, suggesting that the selected beta=0.001 offered little measurable benefit under the evaluated configuration.

Comparative Performance Analysis

The fixed-beta condition is used as the primary reference for evaluating adaptive controllers. The rule-based controller improved the validation score by 0.024193 in absolute value compared with the fixed-beta baseline, equivalent to 2.42 percentage points or 7.46% relative improvement. The MLP improved the score by 0.018145, equivalent to 1.81 percentage points or 5.59% relative improvement. The corrected LSTM benchmark improved the score by 0.042338, equivalent to 4.23 percentage points or 13.06% relative improvement. The zero-KL and fixed-beta scores were almost identical. However, the zero-KL run used two rollouts per prompt whereas the other benchmark conditions used four. Therefore, this near-equivalence should not be interpreted as definitive evidence that beta=0.001 is equivalent to zero regularization.

Analysis of Individual KL-Control Strategies

The zero-KL baseline achieved validation correctness of 0.324600. The fixed-beta condition achieved 0.324597. The rule-based controller achieved 0.348790, the MLP achieved 0.342742, and the corrected LSTM benchmark achieved 0.366935. The RBF controller was implemented using radial basis functions but the preserved result artifact reports failure without a verified validation score. It is therefore excluded from empirical ranking.

Analysis of Adaptive KL Control

The central empirical proposition is that dynamically selecting beta can improve mathematical reasoning performance relative to the selected fixed-beta baseline. The fully completed rule-based and MLP controllers both outperformed fixed-beta, and the corrected LSTM benchmark was the strongest overall result. This pattern is consistent with the hypothesis that adaptive control can be beneficial under the evaluated configuration. The main qualitative difference between the preserved stepwise traces is that the fixed controller remains flat while the rule-based controller progressively lifts beta toward the upper bound as KL rises. The learned controllers are judged primarily through their final validation scores and configuration settings because their per-step beta traces were not preserved as consistently in the archived artifacts.

Research Question Evaluation

The research question asked whether dynamically controlling the KL regularization coefficient beta during GRPO fine-tuning improves mathematical reasoning performance compared with fixed-beta and zero-KL baselines. Under the evaluated configuration, the answer is supported for the performance component. The rule-based controller achieved 0.348790 and the MLP achieved 0.342742 compared with 0.324597 for fixed-beta, while the corrected LSTM benchmark reached 0.366935. The evidence therefore supports the conclusion that adaptive KL control can improve mathematical reasoning performance under the tested conditions. The stability component cannot be established definitively. Detailed per-step loss, KL, beta, and gradient trajectories are not consistently preserved for all controllers, so stability claims are limited to the visible traces.

Hypothesis Evaluation

The key hypotheses were evaluated against the preserved evidence. H1: adaptive KL control improves performance compared with fixed-beta. Supported. H2: adaptive KL control provides more stable policy optimization than zero-KL. Inconclusive. H3: learned controllers outperform heuristic control. Partially supported by the corrected LSTM benchmark, but not by the rule-vs-MLP comparison alone. H4: the LSTM benefits from temporal training-state information compared with a stateless controller. Supported at the final-score level by the corrected benchmark, but not by a complete per-step trajectory archive. These classifications are deliberately restricted to the evidence produced by the experiment and should not be generalized to all models, datasets, or controller configurations.

Discussion of Findings

The principal finding is that adaptive KL control improved validation correctness relative to the selected fixed-beta baseline. The improvement was meaningful but modest for the rule-based and MLP controllers, and larger for the corrected LSTM benchmark. A particularly important result is that the simplest adaptive controller is not automatically the weakest: the heuristic rule outperforms the MLP, while the corrected LSTM benchmark is the strongest final result in the preserved campaign analysis. This demonstrates that additional model complexity is not a guarantee of better adaptive regularization. The deterministic controller provides immediate feedback and requires no controller-parameter optimization, which may be advantageous under a short training horizon. The MLP result nevertheless supports the feasibility of learned beta control. The corrected LSTM benchmark suggests that temporal state can help when the run is fully parsed and the checkpoint sequence is preserved.

Experimental Limitations

Single random seed: all benchmark conditions use seed 42. The fixed trace used here is a smoke run, while the rule trace is the archived benchmark run with preserved beta/KL dynamics. The corrected LSTM result is taken from the campaign analysis and the evaluation logs, not from the stale placeholder JSON. Unavailable RBF result: no verified completed benchmark score is available for the RBF controller. Zero-KL rollout difference: zero-KL used n=2 rollouts per prompt, while the other principal conditions used n=4. Missing training trajectories: per-step KL and beta evolution are not consistently preserved for all controllers. Fixed hyperparameter selection: only beta=0.001 was evaluated as the fixed baseline. Limited computational scope: the experiment uses one small language model, one mathematical reasoning configuration, one principal training schedule, and one random seed.

Chapter Summary

This chapter presented the empirical evaluation of KL-control strategies during GRPO fine-tuning of Qwen2.5-0.5B-Instruct. The main findings are: adaptive KL control improved validation correctness relative to the selected fixed-beta baseline; the rule-based controller achieved 0.348790; the MLP controller achieved 0.342742; the corrected LSTM benchmark achieved 0.366935 and is the strongest preserved result; fixed-beta and zero-KL produced almost identical scores; the rule-based controller showed the clearest beta evolution in the archived traces; and claims about training stability remain limited by partial trajectory preservation. The overall evidence supports the research proposition that dynamic KL control can improve mathematical reasoning performance under the evaluated configuration. At the same

time, the findings should be interpreted as single-seed experimental evidence rather than as universal conclusions about adaptive KL control.